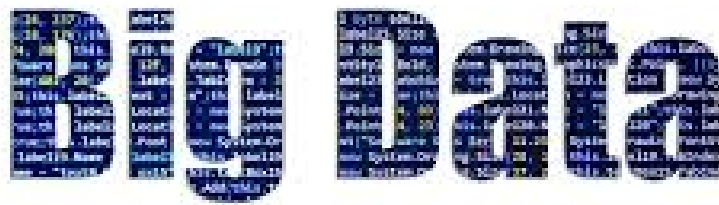


Rapport projet Big Data : ingénierie et traitement (IDAMS 101)

Groupe 5



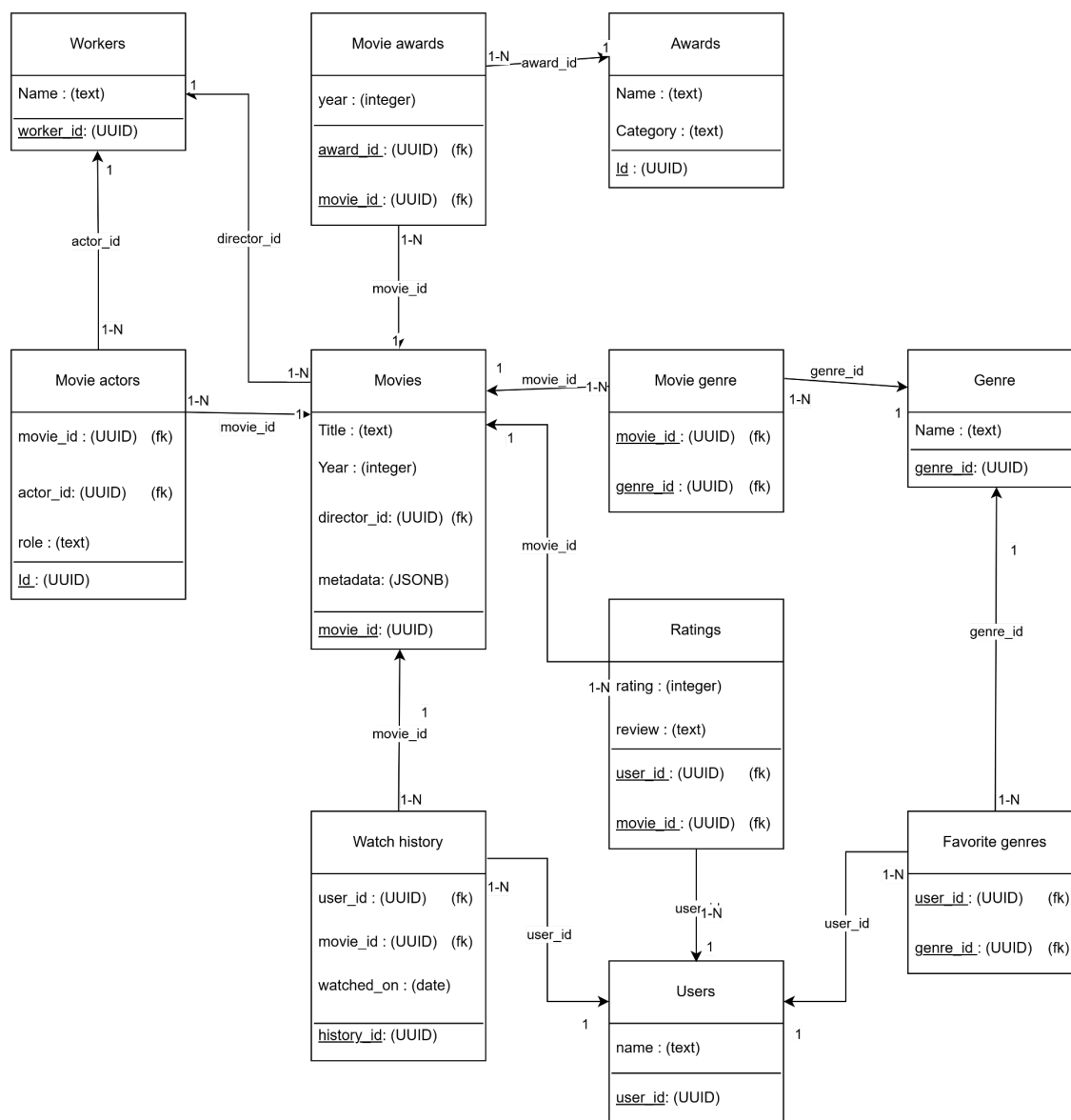
Arnauld MARCHAL
Lynda TCHAKO KADJI
Quentin DE WEE
Benjamin BELLOT



Partie 1 : SQL



1.1 Schéma :



1.2 Liste des contraintes :

En rouge = primary key

En vert = foreign key

En mauve = foreign key ET primary key (les foreign keys forment la primary key)

Awards

- **award_id** : not null
- nom : not null
- catégorie : not null

Nous avons trouvé utile de forcer les informations dans tous les attributs car un award est forcément rattaché à une catégorie et nous pourrions perdre de l'information utile s'il en manquait une. Nous avons hésité pour la catégorie, puis nous en avons conclu qu'il pourrait y avoir le prix du meilleur film mais que sans catégorie, il n'avait pas vraiment de valeur (sans trop nous y connaître dans le domaine des prix pour les films).

Favorite_genres

- **user_id** : not null (fk vers la table users)
- **genre_id** : not null (fk vers la table genre)

Favorite_genres étant une table "bridge", tous ces éléments sont obligatoires étant donné qu'ils servent à relier les informations de 2 tables entre elles.

Genre

- **genre_id** : not null
- **nom** : not null

L'id est bien évidemment non null, et si nous voulons pouvoir observer les informations et les manipuler pour certaines requêtes, le nom est obligatoire également (lister tous les films du genre action par exemple) car l'id est difficile à manipuler par un humain.

Movie_actors

- **id** : not null
- **movie_id** : not null (fk table movies)
- **actor_id** : not null (fk table workers)
- **rôle** : nullable

L'id doit être non-nul. movie_id et actor_id également car le but de la table étant de rattacher les acteurs aux films auxquels ils ont joués, nous sommes obligés d'avoir les 2 foreign keys pour que la ligne soit pertinente dans la table.

Nous avons décidé que le rôle n'était pas obligatoire car un utilisateur peut très bien vouloir connaître la liste des acteurs sans pour autant s'intéresser au rôle et aussi parce que le rôle n'est pas une composante essentielle pour le but premier de la table qui est de relier les acteurs à leur film.

Movie_awards

- **movie_id** : not null (fk table movies)
- **award_id** : not null (fk table awards)
- **année** : nullable

La table permet de relier un film au prix qu'il aurait obtenu, par conséquent, les 2 foreign keys sont obligatoires.

Nous avons jugé l'année non obligatoire car elle n'est pas obligatoire pour remplir le but de la table. De plus, un utilisateur pourrait vouloir connaître les différents prix attribués à un film sans forcément se soucier de l'année d'obtention. Nous avons donc jugé trop contraignant de forcer l'information pour l'année.

Movie_genre

- **movie_id** : not null (fk table movies)
- **genre_id** : not null (fk table genre)

La table Movie_genre sert à relier un film à son genre. Si les 2 composants ne sont pas présents, l'information n'a pas lieu d'être.

Movies

- **movie_id : not null**
- titre : not null
- année : nullable
- **director_id : nullable (fk table Workers)**
- metadata : nullable

Nous avons décidé de ne forcer que l'id du film ainsi que son titre. Il est trop contraignant de supprimer un film uniquement parce qu'on ne connaîtrait pas son année de sortie, son directeur de production, ou parce qu'on aurait pas de données supplémentaires comme une description.

Ratings

- **user_id : not null (fk table users)**
- **movie_id : not null (fk table movies)**
- rating : not null
- review : nullable

La table Ratings permet d'attribuer une note d'un utilisateur à un film. Par conséquent, il est obligatoire de connaître la personne qui attribue la note et de savoir à quel film elle attribue la note et surtout qu'elle est la note. La question du user_id pourrait se poser en se disant qu'elle pourrait ne pas être obligatoire car on pourrait attribuer une note anonyme.

Seulement, si l'on faisait ça, on pourrait imaginer quelqu'un de mal intentionné faire pleins de notes basses pour baisser intentionnellement et artificiellement la note moyenne du film et il serait impossible de trier ces données sans pouvoir connaître l'utilisateur.

Nous avons décidé que la review, elle, n'était pas obligatoire car certaines personnes laissent une note sans message comme sur amazon par exemple.

Users

- **user_id : not null**
- nom : not null

La table Users ne comporte que des éléments non nuls car l'identifiant ne peut être nul, et pour pouvoir manipuler un utilisateur facilement, le nom est obligatoire également

Watch_history

- **history_id : not null**
- **user_id : not null (fk table users)**
- **movie_id : not null (fk table movies)**
- watched_on : nullable

La table Watch_history sert à faire le lien entre les utilisateurs et les films afin de répertorier les films qu'un certain utilisateur aurait déjà vu (historique). Par conséquent, l'identifiant est forcément non nul et les attributs user_id et movie_id se doivent d'être non nuls également afin de pouvoir faire le lien entre les utilisateurs et les films. La date à laquelle le film fut regardé a été jugée non indispensable car cela permet de savoir qu'un utilisateur a déjà vu un film sans savoir quand (à l'image de la barre rouge sous les vidéo youtube par exemple).

Workers

- **worker_id : not null**
- nom : not null

La table Workers répertorie toutes les personnes qui ont participé à un film (acteur et directeur). L'id est obligatoire et pour pouvoir manipuler les travailleurs facilement, leur nom est obligatoire également.

1.3 Précision sur certaines requête de population de la base de donnée :

Certains attributs jugés non vitaux n'apparaissent même pas dans les conditions des requêtes, voici lesquels :

- "metadata" de la table "Movies"
- "review" de la table "ratings"

Ceux-ci n'ont pas de "conditions" quant à leur inclusion dans la table mise à part leur type.

Cela est fait exprès, étant donné qu'ils ne peuvent qu'apporter de l'information superficielle et qui pourrait être non structurée. Étant donné qu'il n'y a pas de format prédéfini pour ces données, il est impossible de les soumettre à une regex par exemple ou à des vérifications sans savoir à quoi est censée ressembler l'information. Nous avons donc fait le choix de les laisser tel quel pour ne pas supprimer des lignes importantes à cause d'une information non vitale.

- L'attribut "watched_on" de la table "Watch_history"

Une remarque uniquement pour spécifier que la date de début (1800-01-01) n'est pas très réaliste mais nous ne savons pas quand la placer. Elle est donc là pour montrer que nous avons pensé à la mettre sans savoir comment la définir précisément.

1.4 Requêtes de création, peuplement et requêtes demandées dans l'énoncé pour SQL :

Toutes les requêtes sont dans le fichier SQL inclus dans le dossier. Elles sont rangées par ordre d'utilisation.

- Création des tables tampon
- Suppression des doublons dans les tables tampons
- Migration des données des tables tampons vers les tables finales
- Migration des données des tables tampons vers les tables rejets (pour vérifier qu'on ne rejette pas trop)
- Requêtes de l'énoncé
- Requête supplémentaire pour SQL

Il n'y a pas de requête de création de table pour les tables finales car celles-ci ont été créées avec l'interface pgAdmin.

1.5 Analyse des performances SQL :

Analyse temporelle :

- Trouver les films avec une note moyenne supérieure à 4.5 :
Temps d'exécution = 4.003 secondes
- Récupérer toutes les informations d'un film avec ses acteurs, son directeur et ses récompenses.
Temps d'exécution = 19.138 secondes.
- Trouver des films non regardés par un utilisateur ayant des acteurs qui ont joué dans des films qu'il a déjà regardés.
Temps d'exécution = 4 minutes 8.442 secondes.
- Requête supplémentaire : Afficher les utilisateurs qui ont regardé le plus de films.
Temps d'exécution = 1.451 secondes.

Analyse de facilité de requêtage :

- Trouver les films avec une note moyenne supérieure à 4.5 :
Requête simple et très courte.
- Récupérer toutes les informations d'un film avec ses acteurs, son directeur et ses récompenses.
et
- Trouver des films non regardés par un utilisateur ayant des acteurs qui ont joué dans des films qu'il a déjà regardés.
Les 2 requêtes sont très longues et complexes avec beaucoup de join, de select et de group by.

Cas d'utilisation approprié :

Les bases de données relationnelles sont plus adaptées aux recherches analytiques utilisant des moyennes et pour comparer sur base de nombres (mean, count, sum,...) les différents attributs. Elles semblent également très bien pour vérifier la cohérence des données.

Partie 2 : MongoDB



mongoDB

2.1 Modèle document

Collections staging

Collection workers_clean

```
{
  workers_clean: {
    _id: ObjectId,

    id: String,
    name: String
  }
}
```

Collection users_clean

```
{
  users_clean: {
    _id: ObjectId,

    id: String,
    name: String
  }
}
```

Collection movies_clean

```
{
  movies_clean: {
    _id: ObjectId,

    id: String,
    title: String,
    year: Number,
    director_id: String,

    rating: Number,
    metadata: Object
  }
}
```

Collection awards_clean

```
{
  awards_clean: {
    _id: ObjectId,

    id: String,
    name: String,
    category: String
  }
}
```

Collection genres_clean

```
{
  genres_clean: {
    _id: ObjectId,

    id: String,
    name: String
  }
}
```

Collection ratings_clean

```
{
  ratings_clean: {
    _id: ObjectId,

    user_id: String,
    movie_id: String,
    rating: Number
  }
}
```

Collection movies_actors_clean

```
{
  movies_actors_clean: {
    _id: ObjectId,

    movie_id: String,
    actor_id: String,
    role: String
  }
}
```

Collection movies_awards_clean

```
{
  movies_awards_clean: {
    _id: ObjectId,

    movie_id: String,
    award_id: String,
    year: Number
  }
}
```

Collection movies_genres_clean

```
{
  movies_genres_clean: {
    _id: ObjectId,

    movie_id: String,
    genre_id: String
  }
}
```

Collection watch_history_clean

```
{
  watch_history_clean: {
    _id: ObjectId,

    user_id: String,
    movie_id: String,
    watched_at: Date
  }
}
```

Collections métier

Collection fiche_film_complet

```
{
fiche_film_complet: {
id: String,
title: String,
year: Number,

director: {
id: String,
name: String
},

actors: [
{
actor_id: String,
actor_name: String,
role: String
}
],

awards: [
{
award_id: String,
award_name: String,
award_category: String,
year: Number
}
],

genres: [String]
}
```

Collection films_bien_notes

```
{
  films_bien_notes: {
    _id: ObjectId,

    movie_id: String,
    title: String,
    year: Number,

    director_id: String,

    note_moyenne: Number,
    nombre_notes: Number
  }
}
```

Collection user_seen_actors

```
{
  user_seen_actors: {
    _id: ObjectId,

    user_id: String,

    actor_ids: [
      String
    ],

    watched_movie_ids: [
      String
    ],

    number_of_seen_actors: Number,
    number_of_seen_movies: Number
  }
}
```

Collection actor_movies

```
{
  actor_movies: {
    _id: ObjectId,
    actor_id: String,

    movie_ids: [
      String
    ],
    number_of_movies: Number
  }
}
```

Collection historique_acteur

```
{
  historique_acteur: {
    _id: ObjectId,

    actor_id: String,
    actor_name: String,

    films: [
      {
        title: String,
        year: Number
      }
    ],

    awards: [
      {
        name: String,
        category: String
      }
    ]
  }
}
```

2.2 Peuplement des collections

2.2.1 Méthodologie générale

La première phase de construction pour notre modèle document MongoDB, consiste en une couche de documents normalisés permettant la préparation et validation de nos données. En effet, nous construisons des collections brutes, semblables au schéma initial, ce qui favorise la cohérence et l'homogénéité des données manipulées dans les différentes requêtes du projet. De plus, cette étape nous permet de garantir la qualité des données, en appliquant des contraintes de validation qui vérifient l'existence des champs obligatoires, éliminent les valeurs nulles ou vides, contrôlent les types de données attendus et nettoient les chaînes de caractères.

Nous appliquons ensuite une phase de consolidation de données dans des collections dénormalisées, permettant d'optimiser l'application de chaque contrainte. En effet, cette étape nécessite l'exécution de nombreux opérateurs **\$lookup** pour reconstruire des vues métier complexes, afin de réduire le coût de ces jointures lors de l'exécution des requêtes. Plus concrètement, nous regroupons toutes les

informations nécessaires à la résolution d'une requête dans un même document, ce qui permet l'exécution rapide et sans jointure.

Enfin, la dernière phase de traitement des requêtes avec l'application d'indicateurs de performance liés au temps d'exécution de chaque opération. Cette phase nous permet de tester et d'optimiser la résolution des requêtes de l'utilisateur final.

2.2.2 Contraintes tables staging “cleans”

Nous reprenons les différentes contraintes de validation établies lors de notre construction de la base de données SQL. Celle-ci se distingue entre des contraintes générales appliquées à chaque collection, et des contraintes spécifiques identifiées pour certaines tables. Ces contraintes permettent la construction de toutes les collections “cleans”.

Contraintes communes

- Vérification de l'existence des champs : { \$exists: true }
- Exclusion des valeurs nulles : { \$ne: null }
- Contrôle du types de données : \$type: "string" ou \$type: "number"
- Suppression du champ _id d'origine : _id: cette opération permet de forcer MongoDB à supprimer l'ancien identifiant de collection, et de recréer un nouvel identifiant pour chaque document. Ces identifiants n'entrent pas en compte dans les liaisons entre collections, car celle-ci sont reliées par les collections métiers tels que movie_id, actor_id, award_id, genre_id, etc. En revanche, cela permet d'éviter tout problème de conflits d'identifiants. Cette contrainte est appliquée à toutes les collections.

Contraintes spécifiques :

- Suppression de chaînes vides : { \$nin: [null, ""] } , pour les collections workers, users, movies(title), awards, genres. Cette opération permet d'éviter la présence de noms ou titres vides.
- Nettoyage des espaces inutiles : \$trim dans les collections workers, users, movies(title), awards, genres, movies_actors(role)

- Limitation de l'année entre 1888 et 2030 : pour la table movies_clean :

```
year: {
  $exists: true,
  $type: "number",
  $gte: 1888,
  $lte: 2030
}
```

- Limitation de la note entre 0 et 5 : pour la table ratings_clean :

```
rating: {
  $exists: true,
  $type: "number",
  $gte: 0,
  $lte: 5
}
```

2.2.3 Requêtes

Requête 1 : *Récupérer toutes les informations d'un film : titre, nom directeur, nom acteur, rôle, award, catégorie award*

Pour cette requête, nous créons une collection fiche_film_complet, que MongoDB rempli progressivement comme suit :

Aggrège les données de la collection movies_clean, et peuple avec :

```
{
  _id: ObjectId(...),
  id: "MOV123",
  title: "Inception",
  year: 2010,
  director_id: "DIR456"
}
```

le **réalisateur**, via director_id relié à workers_clean.id ;
les **acteurs**, via movies_actors_clean puis workers_clean ;
les **récompenses**, via movies_awards_clean puis awards_clean ;
les **genres**, via movies_genres_clean puis genres_clean.

Nous obtenons un document fiche_film_complet avec toutes les informations nécessaires.

Ensuite, on crée un index sur le titre :

```
db.fiche_film_complet.createIndex({ title: 1 });
```

Puis on recherche tous les films ayant le titre :

```
title: "Card"
```

La requête retourne :

- le titre ;
- l'année ;
- le nom du directeur ;
- la liste des acteurs et leurs rôles ;
- les awards et leurs catégories ;
- les genres.

Requête 2 : Trouver les films avec une note moyenne supérieure à 4.5

Pour cette requête, nous créons une collection `films_bien_notes` qui reprend tous les films ayant une note supérieure ou égale à 4.5.

La première étape consiste à regrouper toutes les notes par film, et de calculer la note moyenne :

```
{
  $group: {
    _id: "$movie_id",
    note_moyenne: { $avg: "$rating" },
    nombre_notes: { $sum: 1 }
  }
}
```

Ensuite, nous conservons uniquement les films ayant une note moyenne ≥ 4.5 :

```
{
  $match: {
    note_moyenne: { $gt: 4.5 }
  }
}
```

Enfin, nous allons chercher les informations correspondantes pour chaque film via un `$lookup`

Par exemple :

```
{
  _id: "M10",
  note_moyenne: 4.625,
  movie: [
    {
      id: "M10",
      title: "Inception",
      year: 2010,
      director_id: "D5"
    }
  ]
}
```

Le document devient :

```
{
  movie_id: "M10",
  title: "Inception",
  year: 2010,
  director_id: "D5",
  note_moyenne: 4.63,
  nombre_notes: 4
}
```

En fonction des informations souhaitées à afficher lors de la requête, des jointures supplémentaires peuvent être ajoutées lors de la requête. Afin de ne pas alourdir davantage la collecte de documents dans la collection `films_bien_notes`, nous choisissons de ne pas ajouter d'avantages d'informations liés au film.

Requête 3 : *Trouver des films non regardés par un utilisateur ayant des acteurs qui ont joué dans des films qu'il a déjà regardés*

Cette requête est particulière dans le contexte d'une application en MongoDB, car elle nécessite une recherche sur plusieurs niveaux : partir d'un utilisateur, retrouver ses films vus, retrouver tous les acteurs de ces films, retrouver tous les autres films de ces acteurs, retirer les films déjà vus.

La méthode classique appliquée lors des requêtes précédentes, qui consiste à agréger les résultats dans une collection, s'avère être bien trop lourde en traitement. En effet, on multiplie les combinaisons films x nombres d'acteurs, acteurs x nombres

de films et utilisateurs x nombres de films vus, rendant une collection de document trop volumineuse. Nous avons un exemple concret d'une première tentative pour laquelle nous avons voulu construire une collection reprenant tous les films par utilisateur ayant des acteurs connus par cet utilisateur. Cette collection a dû être stoppée après avoir tourné 88 minutes.

Afin de résoudre ce problème, nous construisons 2 collections distinctes :

- `user_seen_actors` : qui reprend tous les acteurs ayant joué dans un film vu par l'utilisateur. Cette collection permet de réduire significativement le temps de traitement : 2 min 58s

À partir de la collection `watch_history_clean`, une jointure est effectuée avec `movies_actors_clean` afin de récupérer les acteurs associés à chaque film visionné. Les données sont ensuite regroupées par utilisateur pour constituer deux ensembles suivants :

- la liste des acteurs déjà rencontrés (`actor_ids`) ;
- la liste des films déjà visionnés (`watched_movie_ids`).

Des indicateurs statistiques sont également calculés, notamment le nombre d'acteurs et le nombre de films associés à chaque utilisateur.

- `actor_movies` : listant les noms et informations de films pour chaque acteur.

À partir de la collection `movies_actors_clean`, les documents sont regroupés par acteur afin de construire une liste unique de tous les films dans lesquels celui-ci a joué. Le nombre total de films par acteur est également calculé.

Ces 2 collections nécessitent l'application d'une requête dynamique incluant plusieurs jointures avec : `movies_clean` et `workers_clean` pour récupérer titre, année et réalisateur.

La requête inclut également le tri des films par acteur repris dans `movies_actors_clean`, n'ayant pas été vu par l'utilisateur

Ceci impacte la performance d'exécution de la requête, mais permettent de retourner tous les films non vus, pour un nom d'utilisateur donné.

Requête 4 : Introduire le nom d'un acteur et retourne tous les films dans lequel il a joué ainsi que tous les awards gagnés

Une quatrième requête est proposée, permettant d'illustrer la logique documentaire de MongoDB. En effet, cette requête nécessite plusieurs jointures, d'ordinaire lourdes en traitement pour des bases de données relationnelles.

Suivant les mêmes principes évoqués précédemment, une collection dénormalisée historique_acteur est construite, reprenant les documents suivants :

```
{
  historique_acteur: {
    _id: ObjectId,

    actor_id: String,
    actor_name: String,

    films: [
      {
        title: String,
        year: Number
      }
    ],

    awards: [
      {
        name: String,
        category: String
      }
    ]
  }
}
```

Analyse d'index :

Pour cette requête nous ajoutons une comparaison entre l'utilisation de 2 index de recherche.

Le premier **index classique** permet de faire correspondre la recherche à une valeur spécifique, en parcourant tous les documents, et retourne 1 document contenant cet index. Dans notre cas, nous créons un index par nom d'acteur, une fois la recherche lancée, MongoDB retourne 1 document équivalent à l'acteur recherché.

L'inconvénient de cette méthode est qu'elle nous oblige à entrer une valeur correcte, aucun index n'étant retourné si le nom d'acteur ne correspond pas exactement à l'index dans le document : `db.historique_acteur.createIndex({ actor_name: 1 })`

Le deuxième index dit l'index texte, permet de retourner toutes les solutions comprenant entièrement ou partiellement la valeur entrée dans la requête. Cet index fonctionne comme un "moteur de recherche" qui retournerait toutes les options possibles. Concrètement, en cherchant le prénom d'un acteur, par exemple "Lewis", MongoDB retourne tous les documents comprenant Lewis dans le nom de l'acteur :
`db.historique_acteur.createIndex({ actor_name: "text" })`

Dans la partie suivante performance, nous analyserons la résolution sur base de ces 2 index, afin de voir si l'utilisation d'un index ou de l'autre impacte la performance d'exécution.

2.3 Analyses de performances MongoDB

L'analyse des performances de la base MongoDB est basée sur 2 paramètres : d'une part le temps d'exécution d'une requête = **coût d'exécution**, d'autre part le temps d'exécution pour le peuplement d'une collection de documents = **coût de préparation**.

Requête 1 : *Récupérer toutes les informations d'un film : titre, nom directeur, nom acteur, rôle, award, catégorie award*

Peuplement de la collection `fiche_film_complet` :
566 745 ms = 9 min 27 s

Exécution de la requête : en utilisant le titre de recherche "Card", la requête renvoie 86 films en **376 ms = 0,376 s**

Requête 2 : *Trouver les films avec une note moyenne supérieure à 4.5*

Peuplement de la collection `films_bien_notes` :
729235 ms = 12 minutes et 9 secondes

Exécution de la requête : MongoDB renvoie 1571 films en **86 ms = 0,086 s**

Requête 3 : *Trouver des films non regardés par un utilisateur ayant des acteurs qui ont joué dans des films qu'il a déjà regardés*

Nécessite le peuplement de 2 collections (le nombre de collections est un paramètre également à prendre en compte dans l'analyse de performances) :

user_seen_actors : **178 635 ms** = 2 min 59 s

actor_movies : **118 537 ms** = 1 min 59 s

Exécution de la requête : MongoDB renvoie 175 films recommandés sur base d'un nom d'utilisateur "Michelle Anderson" en **1761 ms = 1,761 s**

Requête 4 : *Introduire le nom d'un acteur et retourne tous les films dans lequel il a joué ainsi que tous les awards gagnés*

La collection historique_acteur nécessite : **237 698 ms** = 3 min 58 s

Pour cette requête nous analysons la recherche sur base des 2 indexes expliqués précédemment :

Index classique actor_name: 1

- Temps : **8 ms**
- Documents examinés : 1
- Index keys examinées : 1
- Résultats retournés : 1

Index texte actor_name: 'text'

- Temps : **3 ms**
- Documents examinés : 588
- Index keys examinées : 589
- Résultats retournés : 588

Nous remarquons donc que l'utilisation d'un index texte plutôt qu'un index classique, n'impacte pas la performance d'exécution de notre requête. En fonction de l'utilisation souhaitée, nous pouvons donc passer d'un index à l'autre.

Partie 3 : Neo4j



3) Modélisation et Analyse de la Base de Données Graphe avec Neo4j

Introduction

Dans le cadre de ce projet, une base de données graphe a été implémentée à l'aide de Neo4j afin d'étudier l'intérêt du paradigme orienté graphe pour une plateforme de streaming de films.

Contrairement aux bases relationnelles qui reposent sur des tables et des jointures, Neo4j représente les données sous forme de nœuds et de relations. Cette approche facilite l'exploration des connexions entre les entités et permet de répondre efficacement à des problématiques de recommandation ou d'analyse relationnelle.

Le principal avantage de Neo4j réside dans la gestion et l'exploration de relations complexes. Contrairement aux bases relationnelles qui nécessitent de multiples jointures, Neo4j permet de parcourir directement les liens entre les entités. Il est particulièrement adapté aux systèmes de recommandation, aux réseaux sociaux, à la détection de fraude et à toute application où les relations entre les données sont aussi importantes que les données elles-mêmes.

3.1. Configuration de la BD

a. Scripts de configuration et de peuplement

Le projet a été récupéré depuis le dépôt GitLab fourni puis démarré à l'aide de Docker.

```
git clone
https://gitlab.unamur.be/info-big-data/idasm101-prj/tmpl-prj-bigdata.git

cd tmpl-prj-bigdata

docker compose up -d
```

b. Configuration du chemin des données

Les fichiers CSV ont été placés dans le répertoire d'importation du conteneur Neo4j.

Afin de simplifier les scripts d'importation, un paramètre a été défini :

```
:param basePath => "file:///csv_neo4j/"
```


Ces commandes permettent de télécharger le projet et de lancer automatiquement les conteneurs nécessaires, dont Neo4j.

c. Exemple d'importation des nœuds

```
LOAD CSV WITH HEADERS FROM $basePath + "movies.csv" AS row MATCH
(m:Movie {id: row.id}) MATCH (w:Worker {id: row.director_id}) MERGE
(m)-[:DIRECTED_BY]->(w);
```

MERGE crée uniquement la relation DIRECTED_BY entre le film et son réalisateur en évitant les doublons.

d. Vérification du peuplement

Après l'importation, plusieurs requêtes de contrôle ont été exécutées afin de vérifier la cohérence du graphe.

```
MATCH (n)
RETURN labels(n), count(n);
```

```
MATCH ()-[r]->()
RETURN type(r), count(r);
```

Ces vérifications permettent de confirmer la présence des nœuds et des relations attendues avant l'exécution des requêtes analytiques.

3.2. Modélisation du graphe

a. Nœuds

Les principales entités du domaine ont été modélisées sous forme de nœuds :

- User : utilisateurs de la plateforme
- Movie : films disponibles
- Worker : acteurs et réalisateurs
- Genre : genres cinématographiques
- Award : récompenses obtenues par les films

b. Relations

Les interactions entre ces entités sont représentées par les relations suivantes :

- (:Movie)-[:DIRECTED_BY]->(:Worker)
- (:Worker)-[:ACTED_IN {role}]->(:Movie)
- (:Movie)-[:BELONGS_TO]->(:Genre)
- (:Movie)-[:WON_AWARD {year}]->(:Award)
- (:User)-[:FAVORITE_GENRE]->(:Genre)
- (:User)-[:WATCHED {watched_on}]->(:Movie)
- (:User)-[:RATED {rating, review}]->(:Movie)

Cette modélisation permet de représenter naturellement les interactions entre les utilisateurs, les films et les professionnels du cinéma.

3.3. Contraintes et index

En NoSQL, et plus particulièrement dans Neo4j, l'objectif n'est pas de reproduire l'ensemble des contraintes du modèle relationnel. Les contraintes sont principalement utilisées pour garantir l'unicité des données et améliorer les performances des recherches.

Les contraintes d'unicité suivantes ont été définies :

```
CREATE CONSTRAINT user_id_unique
IF NOT EXISTS
FOR (u:User)
REQUIRE u.id IS UNIQUE;
```

```
CREATE CONSTRAINT worker_id_unique
IF NOT EXISTS
FOR (w:Worker)
REQUIRE w.id IS UNIQUE;
```

```
CREATE CONSTRAINT movie_id_unique
IF NOT EXISTS
FOR (m:Movie)
REQUIRE m.id IS UNIQUE;
```

```
CREATE CONSTRAINT genre_id_unique
IF NOT EXISTS
FOR (g:Genre)
```

```
REQUIRE g.id IS UNIQUE;
```

```
CREATE CONSTRAINT award_id_unique  
IF NOT EXISTS  
FOR (a:Award)  
REQUIRE a.id IS UNIQUE;
```

Ces contraintes empêchent l'apparition de doublons lors des opérations d'importation utilisant la commande MERGE.

En complément, plusieurs index ont été créés afin d'optimiser les recherches :

```
CREATE INDEX movie_title_index  
IF NOT EXISTS  
FOR (m:Movie)  
ON (m.title);
```

```
CREATE INDEX worker_name_index  
IF NOT EXISTS  
FOR (w:Worker)  
ON (w.name);
```

```
CREATE INDEX user_name_index  
IF NOT EXISTS  
FOR (u:User)  
ON (u.name);
```

Les index permettent d'accélérer les opérations MATCH sur les propriétés fréquemment utilisées dans les requêtes analytiques.

3.4. Peuplement de la base

Le peuplement de la base s'est déroulé en plusieurs étapes :

1. Configuration de Neo4j
2. Déclaration du chemin des fichiers CSV
3. Importation des nœuds :
 - Users
 - Workers
 - Movies

- Genres
- Awards
- 4. Importation des relations :
 - DIRECTED_BY
 - ACTED_IN
 - BELONGS_TO
 - WON_AWARD
 - FAVORITE_GENRE
 - WATCHED
 - RATED

L'utilisation de MERGE, combinée aux contraintes d'unicité, a permis d'assurer un import sécurisé.

3.5. Analyse des requêtes

Afin d'évaluer les performances de Neo4j, l'instruction PROFILE a été utilisée.

Cette commande fournit :

- le plan d'exécution ;
- le nombre de lignes parcourues ;
- les opérations réalisées ;
- le coût estimé ;
- les temps d'exécution.

a. Films ayant une note moyenne supérieure à 4.5

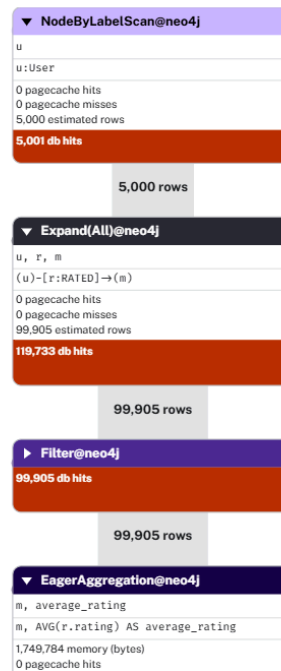
Cette requête identifie les films les mieux évalués par les utilisateurs.

```

PROFILE
MATCH (u:User)-[r:RATED]->(m:Movie)
WITH
  m,
  AVG(r.rating) AS average_rating,
  COUNT(r) AS number_of_ratings
WHERE average_rating > 4.5
RETURN
  m.title AS movie,
  ROUND(average_rating, 2) AS average_rating,
  number_of_ratings
ORDER BY average_rating DESC;
```

Cette requête repose principalement sur des opérations d'agrégation (AVG, COUNT). Elle est réalisable en neo4j mais ne tire donc pas pleinement parti du paradigme graphe, contrairement à PostgreSQL qui est particulièrement adapté à ce type de calcul statistique.

Cypher version: 25
 Planner: COST
 Runtime: SLOTTED
 Total database accesses: 324,554
 Total allocated memory: 1,749,864 bytes
 Total time: 492 ms



b. Récupérer toutes les informations d'un film avec ses acteurs, son directeur et ses récompenses : Cette requête est essentielle pour afficher les fiches complètes des films aux utilisateurs.

Cette requête récupère toutes les informations associées à un film.

PROFILE

```

MATCH (m:Movie {title: "Around politics"})
OPTIONAL MATCH (d:Worker)<-[:DIRECTED_BY]-(m)
OPTIONAL MATCH (a:Worker)-[:role:ACTED_IN]->(m)
OPTIONAL MATCH (m)-[:aw:WON_AWARD]->(award:Award)
RETURN
  m.title AS Titre,
  m.year AS Annee,
  m.metadata AS Metadonnees,
  d.name AS Realisateur,
  COLLECT(DISTINCT {
    acteur: a.name,
  })

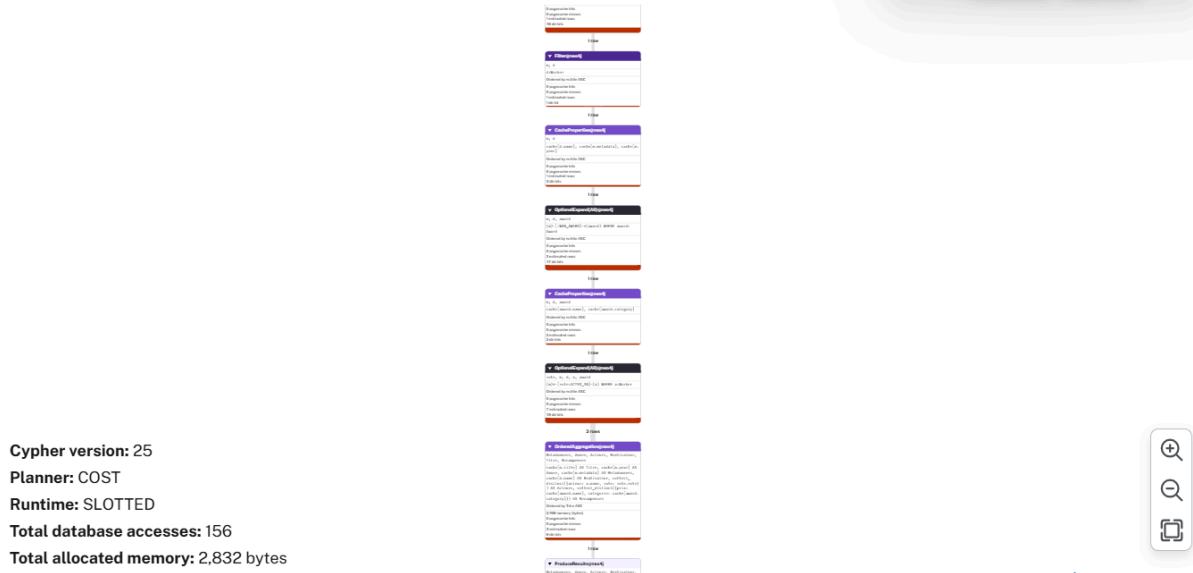
```

```

    role: role.role
  }) AS Acteurs,
  COLLECT(DISTINCT {
    prix: award.name,
    categorie: award.category
  }) AS Recompenses;

```

Cette requête est réalisable en Neo4j et reste lisible grâce aux relations explicites du graphe. Néanmoins, ce type de récupération d'une fiche complète est généralement encore plus naturel dans MongoDB, où toutes ces informations peuvent être regroupées dans un même document enrichi.



The screenshot displays the Neo4j Cypher query editor interface. On the left, the Cypher query is written, and on the right, the execution results are shown as a table. The table contains columns for 'id', 'name', 'role', 'award', and 'category'. The results show a list of actors and their associated awards and categories.

Cypher version: 25
Planner: COST
Runtime: SLOTTED
Total database accesses: 156
Total allocated memory: 2,832 bytes
Total time: 974 ms

[Learn more](#)

- c. **Trouver des films non regardés par un utilisateur ayant des acteurs qui ont joué dans des films qu'il a déjà regardés : Cette requête analytique permet de recommander des films en fonction des préférences des utilisateurs et des relations entre les acteurs et les films.**

PROFILE

```
MATCH (u:User {name: "Alice"})-[:WATCHED]->(seen:Movie)
```

```
MATCH (a:Worker)-[:ACTED_IN]->(seen)
```

```
MATCH (a)-[:ACTED_IN]->(recommended:Movie)
```

```
WHERE NOT EXISTS {
```

```

MATCH (u)-[:WATCHED]->(recommended)
}

RETURN
  recommended.title AS Film_recommande,
  COUNT(DISTINCT a) AS Acteurs_en_commun

ORDER BY Acteurs_en_commun DESC;

```

The screenshot shows a query execution interface. At the top, there are tabs for 'Plan', 'Table', and 'RAW'. The 'Plan' tab is selected, displaying a vertical stack of execution plan steps. To the right of the plan, there are icons for zooming in/out, a search icon, and a download icon. Below the plan, the following statistics are displayed:

- Cypher version: 25
- Planner: COST
- Runtime: SLOTTED
- Total database accesses: 156
- Total allocated memory: 2,832 bytes
- Total time: 738 ms

At the bottom right, there is a 'Learn more' link with an arrow icon.

d. Recommandation de films basée sur les acteurs communs et l'historique de visionnage.

Cette requête identifie les films non encore regardés par un utilisateur mais contenant des acteurs présents dans des films déjà visionnés.

```

PROFILE
MATCH (u:User {name: "Alice"})-[:WATCHED]->(seen:Movie)
MATCH (a:Worker)-[:ACTED_IN]->(seen)
MATCH (a)-[:ACTED_IN]->(recommended:Movie)
WHERE NOT EXISTS {
  MATCH (u)-[:WATCHED]->(recommended)
}
RETURN
  recommended.title AS Film_recommande,
  COUNT(DISTINCT a) AS Acteurs_en_commun
ORDER BY Acteurs_en_commun DESC;

```

Cette requête représente le véritable point fort de Neo4j. Elle consiste à parcourir plusieurs niveaux de relations entre utilisateurs, films et acteurs afin de produire des recommandations personnalisées.

Dans une base relationnelle, cette opération nécessiterait plusieurs jointures et sous-requêtes. Dans Neo4j, elle s'exprime naturellement sous forme de parcours du graphe.

La requête de recommandation précédente constitue également la meilleure démonstration des avantages du paradigme graphe.

Elle permet de parcourir directement les chemins suivants :

Utilisateur → Film vu → Acteur → Nouveau film

Cette approche réduit considérablement la complexité de la requête et améliore sa lisibilité.

MongoDB excelle dans la récupération rapide de documents complets mais n'est pas conçu pour naviguer efficacement à travers plusieurs niveaux de relations.

PostgreSQL permet également de réaliser cette analyse mais nécessite un nombre important de jointures, ce qui augmente la complexité de la requête.

Neo4j offre donc un avantage significatif dès lors que les relations deviennent aussi importantes que les données elles-mêmes.

3.6 Conclusion

L'étude de Neo4j met en évidence les avantages du paradigme graphe pour les applications reposant fortement sur les relations entre les données.

Les contraintes d'unicité et les index ont permis de garantir l'intégrité des données tout en améliorant les performances des recherches.

Les requêtes analytiques simples, telles que les calculs de moyennes, restent davantage adaptées aux bases relationnelles. La récupération d'informations complètes sur un film peut être efficacement réalisée dans MongoDB grâce à son modèle documentaire.

En revanche, les mécanismes de recommandation et les analyses relationnelles complexes constituent le domaine d'excellence de Neo4j. Sa capacité à parcourir

rapidement les relations entre utilisateurs, films et acteurs en fait une solution particulièrement pertinente pour les plateformes de streaming modernes.

Partie finale, comparaison des performances



4. Conclusion générale et comparaison des performances

Au terme de ce projet, l'analyse comparative des trois paradigmes de bases de données met en évidence qu'aucun modèle n'est "totalement supérieur" car leurs performances dépendent fortement de la nature des requêtes, de la structure des données et du coût de préparation (de peuplement) nécessaire avant exécution.

Dans notre cas, PostgreSQL s'est montré le plus pertinent pour les requêtes analytiques reposant sur des agrégations numériques simples, ici en l'occurrence pour celle qui calcule les films ayant une note moyenne supérieure à 4.5. La requête est restée relativement simple à écrire et son temps d'exécution est resté raisonnable par rapport aux autres traitements SQL. En revanche, dès que les besoins impliquent de multiples jointures et des parcours relationnels plus profonds, les performances se dégradent fortement, comme le montre la requête de recommandation, nettement plus coûteuse en temps d'exécution (et en complexité d'écriture).

Quant à MongoDB, son principal avantage réside dans la rapidité d'exécution des requêtes finales lorsque les données ont été préparées à l'avance dans des collections métier dénormalisées. Les résultats obtenus sur la fiche complète d'un film ou sur les films bien notés montrent qu'une fois le coût de préparation "payé", les lectures deviennent beaucoup plus efficaces. En contrepartie, cette performance repose sur une phase préalable de construction parfois longue et coûteuse, qui peut atteindre plusieurs minutes nécessaires pour peupler certaines collections. On a atteint une première tentative abandonnée après 88 minutes pour la requête de recommandation. MongoDB apparaît donc particulièrement adapté lorsque les besoins de consultation sont connus à l'avance et que l'on accepte un coût de prétraitement important pour accélérer l'accès. En revanche, sa rapidité pour donner la solution ajoute une contrainte de place en mémoire qui peut s'avérer non négligeable si la quantité de données est trop élevée étant donné que pour préparer les réponses à l'avance il faut créer une collection à part entière.

Neo4j, de son côté, s'est révélé particulièrement performant et naturel dès lors que la requête repose avant tout sur l'exploration des relations entre entités. La recommandation de films à partir des acteurs communs et de l'historique de visionnage constitue la meilleure illustration de ce point fort : là où PostgreSQL nécessite de nombreuses jointures et où MongoDB doit mettre en place des structures intermédiaires coûteuses, Neo4j permet d'exprimer directement le parcours utilisateur → film vu → acteur → film recommandé. À l'inverse, pour les calculs d'agrégation pure, comme la moyenne des notes, le paradigme graphe

n'apporte pas d'avantage particulier et reste moins pertinent (et en général moins performant comme dans notre cas) qu'un modèle relationnel.

Cette étude confirme donc que le choix d'un système de gestion de données ne doit pas être guidé par une logique de remplacement global, mais par l'adéquation entre un modèle et un usage précis.

Grâce à ce projet on a pu remarquer que pour choisir un type de bases de données il faut prendre en compte à la fois la structure des données mais aussi (et surtout) la façon dont on voudra les utiliser. Pour faire simple, on peut dire que SQL est le plus adapté aux calculs analytiques structurés et aux BD "classiques" relationnelles, MongoDB à la récupération rapide de documents riches préconstruits, et Neo4j à l'analyse relationnelle et à la recommandation. La comparaison des performances montre ainsi qu'une approche multi-modèle est la plus pertinente pour une plateforme de streaming, car elle permet d'exploiter les forces de chaque paradigme selon le type de traitement demandé. En situation réelle, l'idéal ne serait pas de choisir un seul modèle pour tout le système, mais de combiner plusieurs paradigmes, chacun étant utilisé pour le type de données ou de requêtes qu'il gère le plus efficacement et éviter leurs faiblesses respectives qui peuvent parfois faire la différence.

Annexe : travail sur neo4j



Les requêtes et les performances

profile permet d'avoir le plan d'exécution, le nombre de lignes parcourues, les opérations effectuées, le coût estimé, le temps d'exécution

1. **Trouver les films avec une note moyenne supérieure à 4.5 : Cruciale pour proposer aux utilisateurs les films les mieux notés !**

PROFILE

MATCH (u:User)-[r:RATED]->(m:Movie)

WITH

m,
AVG(r.rating) AS average_rating,
COUNT(r) AS number_of_ratings

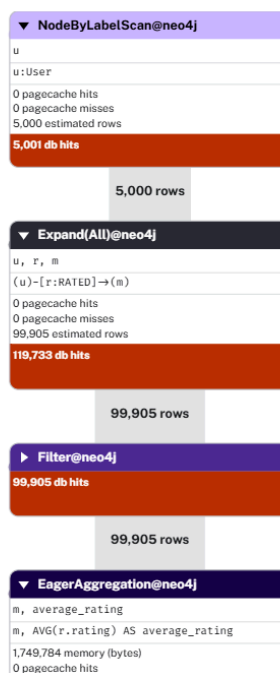
WHERE average_rating > 4.5

RETURN

m.title AS movie,
ROUND(average_rating, 2) AS average_rating,
number_of_ratings

ORDER BY average_rating DESC;

Cypher version: 25
Planner: COST
Runtime: SLOTTED
Total database accesses: 324,554
Total allocated memory: 1,749,864 bytes
Total time: 492 ms



2. **Récupérer toutes les informations d'un film avec ses acteurs, son directeur et ses récompenses : Cette requête est essentielle pour afficher les fiches complètes des films aux utilisateurs.**

PROFILE

MATCH (m:Movie {title: "Around politics"})

OPTIONAL MATCH (d:Worker)<-[:DIRECTED_BY]-(m)

OPTIONAL MATCH (a:Worker)-[:role:ACTED_IN]->(m)

OPTIONAL MATCH (m)-[:aw:WON_AWARD]->(award:Award)

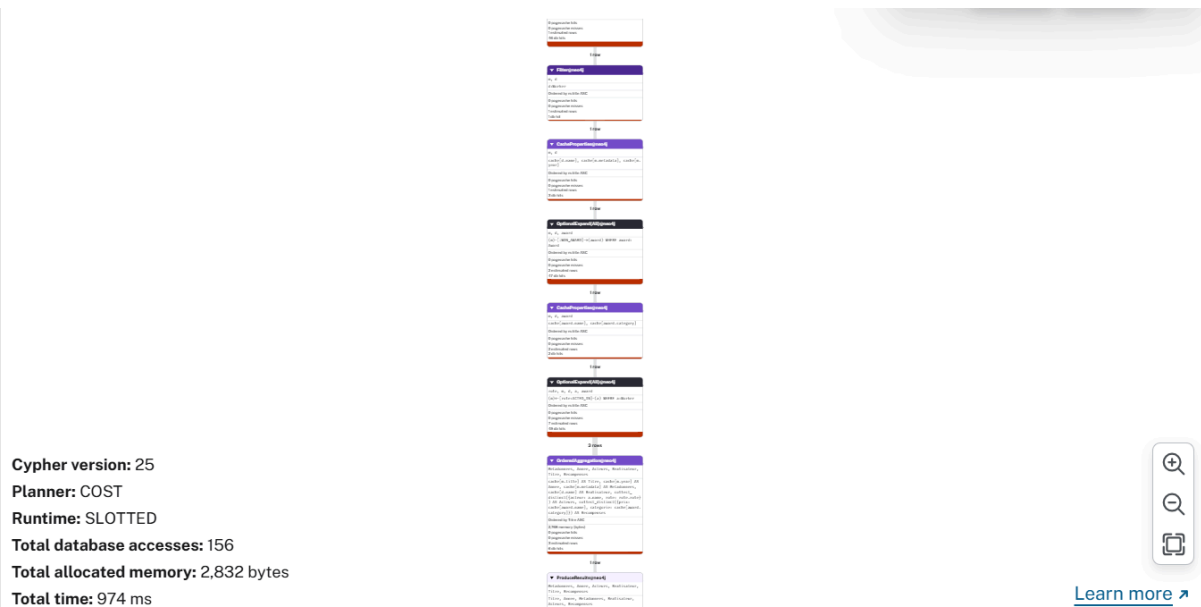
RETURN

m.title AS Titre,
m.year AS Annee,
m.metadata AS Metadonnees,

d.name AS Realisateur,

COLLECT(DISTINCT {
 acteur: a.name,
 role: role.role
}) AS Acteurs,

COLLECT(DISTINCT {
 prix: award.name,
 categorie: award.category
}) AS Recompenses;



3. **Trouver des films non regardés par un utilisateur ayant des acteurs qui ont joué dans des films qu'il a déjà regardés : Cette requête analytique permet de recommander des films en fonction des préférences des utilisateurs et des relations entre les acteurs et les films.**

PROFILE

MATCH (u:User {name: "Alice"})-[:WATCHED]->(seen:Movie)

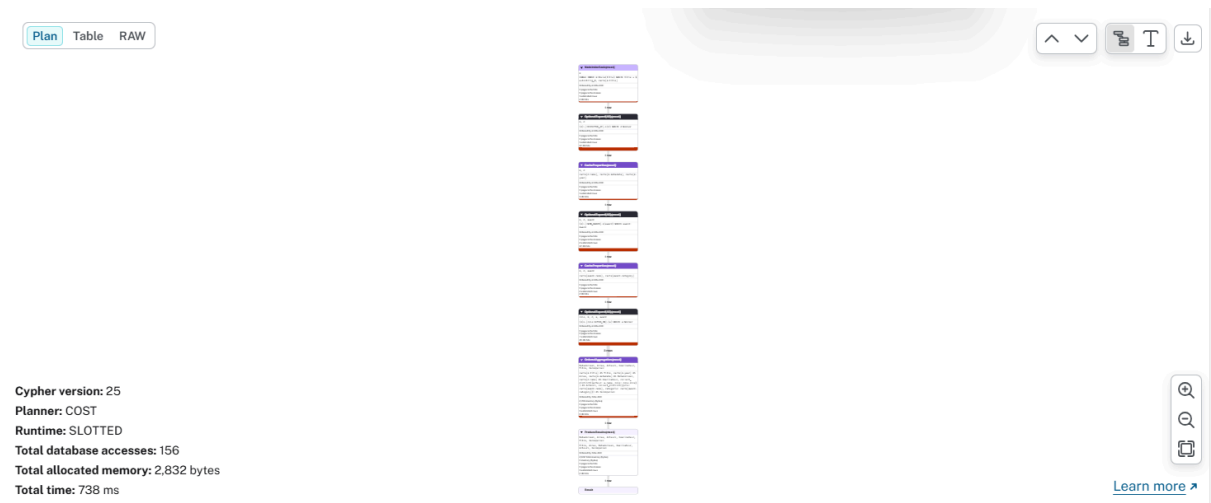
MATCH (a:Worker)-[:ACTED_IN]->(seen)

MATCH (a)-[:ACTED_IN]->(recommended:Movie)

```
WHERE NOT EXISTS {
  MATCH (u)-[:WATCHED]->(recommended)
}
```

```
RETURN
    recommended.title AS Film_recommande,
    COUNT(DISTINCT a) AS Acteurs_en_commun
```

```
ORDER BY Acteurs_en_commun DESC;
```

4. Recommandation de films basée sur les acteurs communs et l'historique de visionnage.

MATCH (u:User {name: "Alice"})-[:WATCHED]->(seen:Movie)

```
WHERE NOT EXISTS {
    MATCH (u)-[:WATCHED]->(recommended)
}
```

recommended.title AS Film_recommande,

Pourquoi ce n'est pas le fort de MongoDB?:MongoDB est excellent pour récupérer rapidement un document complet,mais lorsqu'il faut traverser plusieurs niveaux de relations,ce n'est pas son usage principal.

Cette requête illustre l'avantage principal de Neo4j : l'exploration des relations complexes. La recommandation est obtenue en parcourant directement le graphe entre utilisateurs, films et acteurs. Ce type d'analyse relationnelle nécessite de nombreuses jointures dans une base relationnelle alors qu'il est exprimé naturellement sous forme de parcours dans Neo4j. Cette approche est particulièrement adaptée aux moteurs de recommandation utilisés par les plateformes de streaming.